

Scenario 03 — NetLink ISP Customer Portal

(Digital infrastructure)

Deliberately vulnerable Node.js + PostgreSQL portal for the CyberFort cyber range. Trainees use CyberFort to scan it, demonstrate a JWT forgery from a leaked source secret, complete an ISO/IEC 27001:2022 gap assessment, and produce a PDF gap report.

What's in the box

```
03-netlink-isp-digital-infra/
├─ README.md           ← this file
├─ docker-compose.yml  ← api + postgres
├─ api/                ← Node.js Express + EJS (intentionally vulnerable)
│   ├── Dockerfile
│   ├── package.json    ← pinned to known-vulnerable npm packages
│   ├── server.js
│   └── src/
│       ├── config.js    ← hardcoded JWT secret, DB password, API key
│       ├── db.js
│       ├── auth.js
│       └── routes/
│           ├── auth.js
│           ├── customers.js ← VULN: SQL injection
│           ├── invoices.js  ← VULN: IDOR
│           └── admin.js     ← VULN: diagnostics endpoint echoes secrets
│   └── views/
│       ├── login.ejs
│       └── dashboard.ejs
├─ db/
│   └── init.sql         ← seed users, customers, invoices
└─ docs/
    ├── TRAINEE_HANDBOOK.md
    ├── INSTRUCTOR_GUIDE.md
    └── ISO27001_CONTROL_MAPPING.md
```

Topology

- **VM1** — CyberFort (:5173 , :8000 , scanners on :8010-:8013).
- **VM2** — this scenario. Exposes **3000** (Node.js portal) and **5432** (PostgreSQL — deliberately).

Deploy on VM2

```
git clone <repo-url> /srv/netlink-scenario
cd /srv/netlink-scenario/03-netlink-isp-digital-infra
docker compose up -d --build
docker compose ps      # both services Up
curl http://localhost:3000/healthz
```

What the trainee should see

- Nmap reports **3000/http** and **5432/postgresql** open.
- ZAP active scan flags SQL injection on `/api/customers/search`, IDOR on `/api/invoices/:id`, and missing cookie flags.
- Semgrep flags hardcoded JWT secret, hardcoded DB password, hardcoded API key in `src/config.js`.
- OSV flags Express 4.16.0, jsonwebtoken 8.5.0, lodash 4.17.20, ejs 3.1.5, body-parser 1.18.0, pg 8.7.1.
- A forged admin JWT minted from the hardcoded `JWT_SECRET` opens `/api/admin/diagnostics`, which returns the **plaintext** DB password and the provisioning API key.

Full walkthrough in [docs/TRAINEE_HANDBOOK.md](#).

Safety

Deliberately vulnerable. Run only on an isolated cyber-range network.